

Roter Faden für die mündliche Prüfung

Vom trainierten RL-Agenten zum verlässlichen eingebetteten System

Empfohlener Titel

„Vom Webots-Modell zum autonomen Rennroboter: Sim-to-Real als Aufgabe der Embedded-Softwareentwicklung“

Zentrale Leitfrage

Welche technischen und methodischen Schritte sind nötig, damit meine in der Simulation trainierte PPO-Policy auf realer Hardware rechtzeitig, ressourcenschonend und verlässlich ein Fahrzeug steuern kann?

Kernthese

Die RL-Policy ist noch kein eingebettetes System. Sie ist lediglich eine Komponente innerhalb eines zeitkritischen, hardwareabhängigen und fehlertoleranten Regelkreises.

Diese These ist der rote Faden. Du stellst daher nicht einfach „mein RL-Projekt“ vor, sondern verwendest das Projekt als **Fallstudie**, an der du den gesamten Weg der Embedded-Softwareentwicklung erklärst:

1. funktionierende PC-Simulation,
2. Analyse der Sim-to-Real-Lücke,
3. Entwurf einer Zielarchitektur,
4. Auswahl und Anbindung der Hardware,
5. zeitlich deterministische Ausführung,
6. Deployment der ML-Policy,
7. systematische Verifikation,
8. Fehlertoleranz und sicherer Betrieb.

Damit kannst du fast alle vier Vorlesungsblöcke sinnvoll verbinden, ohne wahllos Begriffe aufzuzählen.

1. Dramaturgie der Präsentation



Der Clou: **Jedes Kapitel beantwortet ein konkretes Problem**, das beim Übergang in die reale Welt entsteht. Der Vorlesungsstoff liefert jeweils die Lösung.

Der eigentliche rote Faden

Kapitel 1 – Was funktioniert bereits in der Simulation?

Erzählziel

Du zeigst zunächst knapp, was du entwickelt hast und warum es ein sinnvoller Ausgangspunkt ist.

Das Projekt besitzt eine sequenzielle Vier-Stufen-Pipeline:

1. **Perception:** LiDAR, IMU und Kamera einlesen,
2. **Estimation:** Initiallokalisierung, ICP und Hinderniskarte,
3. **Planning:** PPO-Policy berechnet Geschwindigkeit und Lenkwinkel,
4. **Control:** Ackermann-Geometrie und Motoransteuerung.

Die Policy erhält einen **15-dimensionalen normalisierten Zustandsvektor** und liefert zwei kontinuierliche Aktionen: Änderung der Lenkung und Änderung der Geschwindigkeit. Das Training erfolgt als zweistufiges Curriculum: zunächst sicheres Spurhalten, anschließend höhere Geschwindigkeit und dynamische Hindernisvermeidung. Für die Inferenz wird die trainierte PyTorch-Policy als ONNX-Modell exportiert. Diese Punkte sind im [Projekt-README](#), im [RL-Trainingsleitfaden](#) und in der [Architekturbeschreibung](#) dokumentiert.

Gute Formulierung in der Prüfung

„Mein derzeitiger Stand ist eine ausführbare Software-in-the-Loop-nahe Lösung: Strecke, Sensorik, Fahrzeugdynamik und Aktorik werden in Webots simuliert, während bereits der spätere Wahrnehmungs-, Lokalisierungs-, Planungs- und Kontrollablauf ausgeführt wird. Nun stellt sich die Embedded-Frage: Was davon kann unverändert auf reale Hardware übertragen werden und was muss neu entworfen werden?“

Vorlesungsbezug

- Softwarearchitektur und Separation of Concerns
- Super Loop
- MiL/SiL als Ausgangspunkt
- Zustandsvektor und Edge-AI-Inferenz

Wichtige Abgrenzung

Bezeichne den Stand nicht unkritisch als vollständiges klassisches SiL, weil deine Software nicht vollständig aus einem Modell generiert wurde. Sage besser:

„Der Aufbau entspricht funktional einer SiL-nahen Stufe: Die reale Hardware fehlt, aber die ausführbare Steuerungssoftware arbeitet bereits gegen ein simuliertes Strecken- und Fahrzeugmodell.“

Kapitel 2 – Wo liegt die Sim-to-Real-Lücke?

Erzählziel

Jetzt entsteht das eigentliche Problem. In der Simulation sind Zeit, Sensorwerte, Aktorik und Rechenressourcen kontrollierbar. Auf dem realen Fahrzeug nicht.

Konkrete Lücken deines Projekts

2.1 Simulationswissen statt realer Messung

Der aktuelle Webots-Controller liest die Fahrzeuggeschwindigkeit direkt über den Supervisor beziehungsweise die simulierte Physik aus. In der Realität muss diese Größe beispielsweise aus Encodern, IMU und Zustandsbeobachtung geschätzt werden. Das ist im [aktuellen Hauptcontroller](#) sichtbar.

Vorlesungsbrücke: Sensorik, ADC, Timer/Counter, Interrupts, Zustandsabschätzung.

2.2 Idealisierte Sensorik

Reale Sensoren besitzen:

- Rauschen und Bias,
- Quantisierung,

- begrenzte Abtastraten,
- Messausfälle,
- zeitliche Verzögerungen,
- unterschiedliche Zeitstempel,
- elektromagnetische und mechanische Störungen.

Die Policy könnte deshalb auf Beobachtungen treffen, die sie im Training nie gesehen hat.

Lösungsansätze: Domain Randomization, Rauschmodelle, Verzögerungen, Sensor-Dropouts und Parameterstreuung bereits während des Trainings simulieren.

2.3 Nichtideale Aktorik

Reale Motoren und Servos besitzen Totzone, Sättigung, Spiel, Schlupf, Versorgungsspannungsabhängigkeit und Verzögerung. Die aktuelle Kontrolle begrenzt zwar Geschwindigkeit und Lenkwinkel und berechnet eine Ackermann-Lenkgeometrie, übergibt die Werte jedoch direkt an simulierte Geräte. Siehe [Control-Modul](#).

Vorlesungsbrücke: PWM über Timer, GPIO, Treiber/HAL, Regelkreis, Deadline.

2.4 Unbegrenzter PC versus begrenzte Zielhardware

Python, NumPy, OpenCV und ONNX Runtime laufen komfortabel auf dem PC. Auf einem Mikrocontroller sind Flash, SRAM, Rechenzeit und Leistungsaufnahme begrenzt.

Der wichtige Befund lautet:

Nicht unbedingt die Policy ist das größte Problem, sondern die gesamte Perception- und Estimation-Pipeline.

Die reine Policy besitzt nach der dokumentierten Architektur ungefähr **18.818 Parameter**. Das sind als Float32 grob **74 KiB Gewichte** und rund **18.560 Multiply-Accumulate-Operationen pro Inferenz**. Bei 10 Hz ist die Netzinferenz grundsätzlich überschaubar. OpenCV, ICP, LiDAR-Verarbeitung, Objektdaten, Laufzeitbibliothek und Zwischenpuffer erhöhen den tatsächlichen Ressourcenbedarf jedoch deutlich. Die Netzarchitektur und die 10-Hz-Regelrate stehen in der [Konfiguration](#); der ONNX-Export enthält nur den Actor-Pfad der Policy, siehe [Exportskript](#).

Übergang zum nächsten Kapitel

„Bevor ich Hardware auswähle, muss ich deshalb aus der Sim-to-Real-Lücke messbare Anforderungen ableiten.“

Kapitel 3 – Anforderungen an das reale eingebettete System

Erzählziel

Du wechselst von „es fährt in Webots“ zu einer ingenieurmäßigen Spezifikation.

Funktionale Anforderungen

- LiDAR, IMU, Kamera und gegebenenfalls Encoder einlesen,
- Fahrzeugpose und Geschwindigkeit schätzen,
- Hindernisse erkennen und verfolgen,
- 15 Eingangsmerkmale reproduzierbar berechnen,
- ML-Policy ausführen,
- Lenkservo und Antrieb ansteuern,
- Kalibrierung, Start, Fahrbetrieb und Stopp verwalten,
- Fehler erkennen und in einen sicheren Zustand wechseln.

Nichtfunktionale Anforderungen

- definierte Kontrollfrequenz,
- begrenzte Ende-zu-Ende-Latenz,
- Einhaltung von Speicher- und Leistungsbudgets,
- reproduzierbares Verhalten,
- Wartbarkeit und Portabilität,
- Robustheit gegenüber Sensor- und Softwarefehlern.

Echtzeitanforderung aus deinem Projekt

Die Konfiguration sieht **10 Hz Kontrollfrequenz** vor. Daraus folgt eine Periode und zunächst auch eine Ende-zu-Ende-Deadline von:

Innerhalb dieses Budgets müssen mindestens Wahrnehmung, Schätzung, Beobachtungsbildung, Inferenz und Ausgabe abgeschlossen werden.

Eine gute zeitliche Zerlegung wäre beispielsweise:

Verarbeitungsschritt	Zu prüfendes Budget
Sensordatenübernahme und Synchronisation	15 ms
Lokalisierung und Mapping	40 ms
Beobachtungsvektor und Policy-Inferenz	15 ms
Aktorberechnung und Kommunikation	10 ms
Reserve für Jitter und Interrupts	20 ms

Das sind zunächst **Entwurfsbudgets**, keine bereits gemessenen Werte. In der Implementierung müssen anschließend BCET, durchschnittliche Laufzeit, WCET-Näherung und Jitter gemessen werden.

Harte oder weiche Echtzeit?

Eine differenzierte Antwort wirkt stärker als „das Auto ist harte Echtzeit“:

- Die optimale RL-Trajektorie ist eher **weiche oder feste Echtzeit**: Ein verspäteter Zyklus verschlechtert zunächst die Fahrqualität.
- Die unmittelbare Aktorbegrenzung, Kollisionsreaktion oder ein Not-Stopp besitzen deutlich strengere Anforderungen.
- Deshalb sollte eine nichtdeterministische ML-Komponente niemals allein für die Sicherheitsreaktion verantwortlich sein.

Kapitel 4 – Welche Zielarchitektur eignet sich?

Erzählziel

Du vergleichst nicht nur Controllerdatenblätter, sondern leitest die Architektur aus den Anforderungen ab.

Drei mögliche Varianten

Variante A – Embedded Linux auf einem leistungsfähigen Rechner

Beispielidee: ARM-basierter Single-Board-Computer mit Linux.

Vorteile:

- Python/C++ sowie ONNX Runtime relativ leicht übertragbar,
- genügend Speicher für OpenCV, ICP und Mapping,
- schnellster Weg zu einem ersten realen Prototyp.

Nachteile:

- weniger deterministische Latenzen,
- größerer Energiebedarf,
- längere Bootzeit,
- Linux allein garantiert keine harte Echtzeit.

Vorlesungsbezug: Embedded Linux, Buildroot oder Yocto, Device Tree, PRE-EMPT_RT.

Variante B – Vollständige Umsetzung auf einem Mikrocontroller

Beispielidee: ESP32-S3 oder leistungsfähiger Cortex-M.

Vorteile:

- geringer Energiebedarf,
- kurze Bootzeit,
- gute Kontrolle über Timing und Peripherie,
- FreeRTOS und direkte Hardwareanbindung.

Nachteile:

- Python-, OpenCV- und ONNX-Runtime-Pipeline nicht direkt übertragbar,
- Quantisierung und C/C++-Portierung nötig,
- SRAM kann besonders für LiDAR, Kamera und Mapping knapp werden.

Der ATmega328 wäre mit 2 KB SRAM bereits für die Policygewichte ungeeignet. Beim STM32F401 mit 96 KB SRAM könnten die reinen Gewichte theoretisch gerade in die Größenordnung passen, aber Laufzeit, Aktivierungen, Stacks, Sensordaten und Lokalisierung benötigen zusätzlichen Speicher. Der ESP32-S3 bietet laut [Vorlesungsglossar](#) 512 KB SRAM, zwei Kerne und ML-orientierte Vektorerweiterungen und wäre damit plausibler – dennoch müsste die Gesamtpipeline genau profiliert werden.

Variante C – Heterogene Architektur als Empfehlung

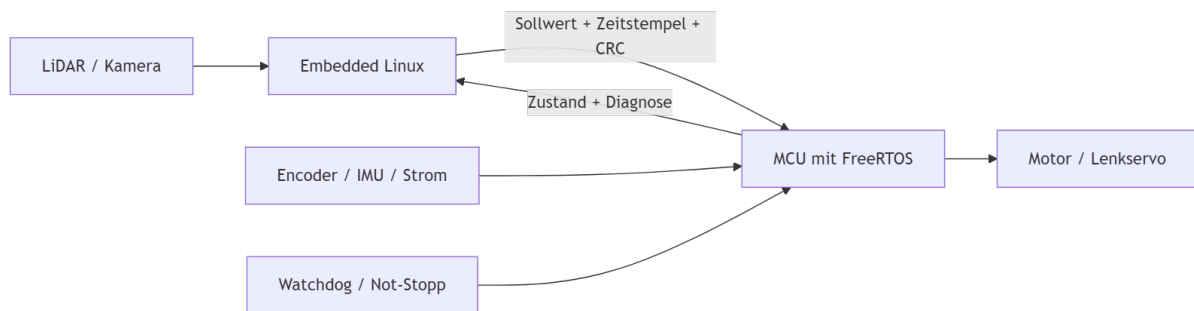
Leistungsrechner / Embedded Linux:

- Kamera und LiDAR,
- ICP und Hinderniskarte,
- Beobachtungsvektor,
- RL-Inferenz,
- Diagnose und Logging.

Mikrocontroller / FreeRTOS:

- Encoder und IMU zeitgenau erfassen,
- Servo- und Motor-PWM,
- Strom-, Spannungs- und Grenzwertüberwachung,
- Watchdog und Not-Stopp,
- sichere Begrenzung der vom RL-Rechner angeforderten Aktionen.

Kommunikation: beispielsweise UART oder CAN mit Sequenznummer, Zeitstempel und Prüfsumme.



Warum diese Variante besonders prüfungstauglich ist

Sie verbindet fast den gesamten Stoff:

- **AMP-artige Aufgabenteilung:** leistungsfähige und echtzeitkritische Teile werden getrennt,
- **Embedded Linux** für komplexe Algorithmen,
- **FreeRTOS** für deterministische Peripherie und Sicherheitsfunktionen,
- **UART/CAN** als Kommunikationsschnittstelle,
- **HAL und BSP** zur Entkopplung konkreter Hardware,
- **Watchdog und Fail-Safe** für Dependability.

Wichtig: Du musst nicht behaupten, dass genau diese Hardware bereits existiert. Präsentiere sie als **begründeten Zielentwurf**.

Kapitel 5 – Von der Super Loop zur Echtzeitsoftware

Ausgangspunkt

Der aktuelle Controller arbeitet im Kern wie eine **Super Loop**: Webots-Schritt, Perception, Estimation, Planning und Control werden sequenziell ausgeführt. Das ist einfach und nachvollziehbar, aber ein langsamer Verarbeitungsschritt verzögert alle folgenden Stufen.

Mögliche RTOS-Zerlegung

Task	Beispielperiode	Priorität	Kommunikation
Safety/Not-Stopp	ereignisgetrieben oder 1–5 ms	sehr hoch	Interrupt/Semaphor
Motor- und Servoregelung	5–10 ms	hoch	Queue mit letztem Sollwert
IMU/Encoder-Erfassung	5–10 ms	hoch	Queue oder Ringpuffer
LiDAR/Perception	sensorgegeben	mittel	DMA + Queue
Lokalisierung/Mapping	50–100 ms	mittel	Snapshot/Queue
RL-Inferenz	100 ms	mittel	Queue
Logging/Telemetrie	500–1000 ms	niedrig	Queue

Vorlesungsbegriffe, die natürlich passen

- **Task und TCB**: jede Funktionseinheit besitzt eigenen Kontext und Stack.
- **SysTick**: Zeitbasis für den FreeRTOS-Scheduler.
- **Interrupt/ISR**: Datenübernahme oder Not-Stopp; ISR kurz halten und Verarbeitung an einen Task delegieren.
- **DMA**: Sensordaten oder Kommunikationsframes ohne dauernde CPU-Beteiligung übertragen.
- **Queue**: Messwerte und Sollwerte ohne ungeschützte gemeinsame Variablen übergeben.
- **Semaphor**: ISR signalisiert, dass neue Daten bereitstehen.
-

Mutex: nur für wirklich gemeinsam genutzte Ressourcen, beispielsweise einen Bus oder Diagnoseausgang.

- **Prioritätsinversion:** Safety-Task darf nicht auf einen von einem Logging-Task gehaltenen Mutex warten; PIP/PCP oder bessere Ressourcenaufteilung verwenden.
- **Rate Monotonic Scheduling:** kurze periodische Safety- und Regelungsaufgaben erhalten höhere Prioritäten als langsame Diagnoseaufgaben.
- **WCET und Deadline:** nicht nur mittlere Inferenzzeit messen, sondern Worst-Case-Verhalten betrachten.

Kritischer Punkt

„Mehr Tasks bedeuten nicht automatisch bessere Echtzeit.“

Nebenläufigkeit erzeugt Kontextwechsel, Synchronisation, mögliche Race Conditions und Prioritätsprobleme. Wenn die 100-ms-Deadline mit einer sauberen Super Loop nachweisbar eingehalten wird, kann diese weiterhin sinnvoll sein. Ein RTOS wird dann interessant, wenn unterschiedliche Raten, blockierende I/O oder unabhängige Sicherheitsreaktionen auftreten.

Kapitel 6 – Hardwareabstraktion und Portierung

Erzählziel

Die Algorithmen sollen nicht direkt von Webots-Objekten abhängen.

Der aktuelle Code ruft beispielsweise Webots-Gerätefunktionen für Sensoren und Motoren auf. Für die Portierung sollte zwischen **Anwendungslogik** und **Hardwarezugriff** eine Schnittstelle eingeführt werden.

Sinnvolle Schichten

```
RL-/Autonomie-Anwendung
↓
Perception - Estimation - Planning - Control
↓
plattformunabhängige Sensor-/Aktor-Interfaces
↓
HAL
↓
BSP und konkrete Treiber
```



LiDAR, IMU, Encoder, Kamera, Motor und Servo

Nutzen

- Webots und reale Hardware implementieren dieselben Interfaces.
- Algorithmen können auf dem PC getestet werden.
- Ein Boardwechsel betrifft hauptsächlich BSP und Treiber.
- Die RL-Policy bleibt von UART-, SPI-, I²C- und GPIO-Details getrennt.

Hardwarebegriffe mit konkretem Projektbezug

- **GPIO:** Not-Stopp, Status-LED, Enable-Leitungen.
- **Timer/Counter:** Encoderzählung, periodische Regelung und PWM.
- **ADC:** Batterie-, Strom- oder analoge Sensorspannung messen.
- **UART:** Debugging oder einfacher Sensor-/Rechner-Link.
- **SPI:** schnelle lokale Sensoren oder Speicher.
- **I²C:** IMU und langsamere Sensorperipherie.
- **CAN:** robuste Kommunikation zwischen Rechner und Motor-/Safety-Controller.
- **Flash:** Firmware und quantisiertes Modell.
- **SRAM:** Stacks, Sensordaten, Aktivierungen und Kartenpuffer.
- **EEPROM/Flash-Datenbereich:** Kalibrierwerte und Konfiguration.
- **FPU:** Float32-Lokalisierung und gegebenenfalls Netzinferenz beschleunigen.
- **MPU:** Tasks beziehungsweise Speicherbereiche gegeneinander schützen.

RISC/CISC, Harvard/Von-Neumann und Pipelining kannst du bei der Begründung der Plattform kurz erwähnen, aber sie sollten nicht zum Hauptstrang werden.

Kapitel 7 – Wie wird die Policy zu Edge AI?

Aktueller Stand

Die PPO-Policy wird auf dem PC trainiert und für die Inferenz nach ONNX exportiert. Das ist bereits die richtige Trennung:

- **Training:** leistungsfähiger PC,
- **Deployment:** nur deterministische Inferenz auf dem Zielsystem.

Mögliche Embedded-Schritte

1. Eingangsnormalisierung exakt einfrieren.
2. PyTorch- und ONNX-Ausgaben mit festen Testvektoren vergleichen.
3. Zielruntime auswählen: ONNX Runtime auf Linux oder TFLite Micro/C-Code auf MCU.
4. Float32-Modell profilieren.
5. Post-Training-Quantisierung auf Int8 untersuchen.
6. Speicherbedarf, Latenz und numerische Abweichung messen.
7. Reale Fahrleistung mit derselben Testsuite vergleichen.

Quantisierung

Quantisierung reduziert Modellgröße und Rechenaufwand. Aus ungefähr 74 KiB Float32-Gewichten könnten bei reiner Int8-Speicherung grob 18–19 KiB Gewichte werden. Zusätzlich werden jedoch Quantisierungsparameter, Aktivierungspuffer und Runtime-Code benötigt.

Die entscheidende Frage ist nicht nur:

„Ist das quantisierte Modell kleiner?“

sondern:

„Bleiben Lenk- und Geschwindigkeitsaktionen innerhalb einer akzeptablen Abweichung und fährt der Roboter weiterhin stabil?“

Sicherheitskäfig um die Policy

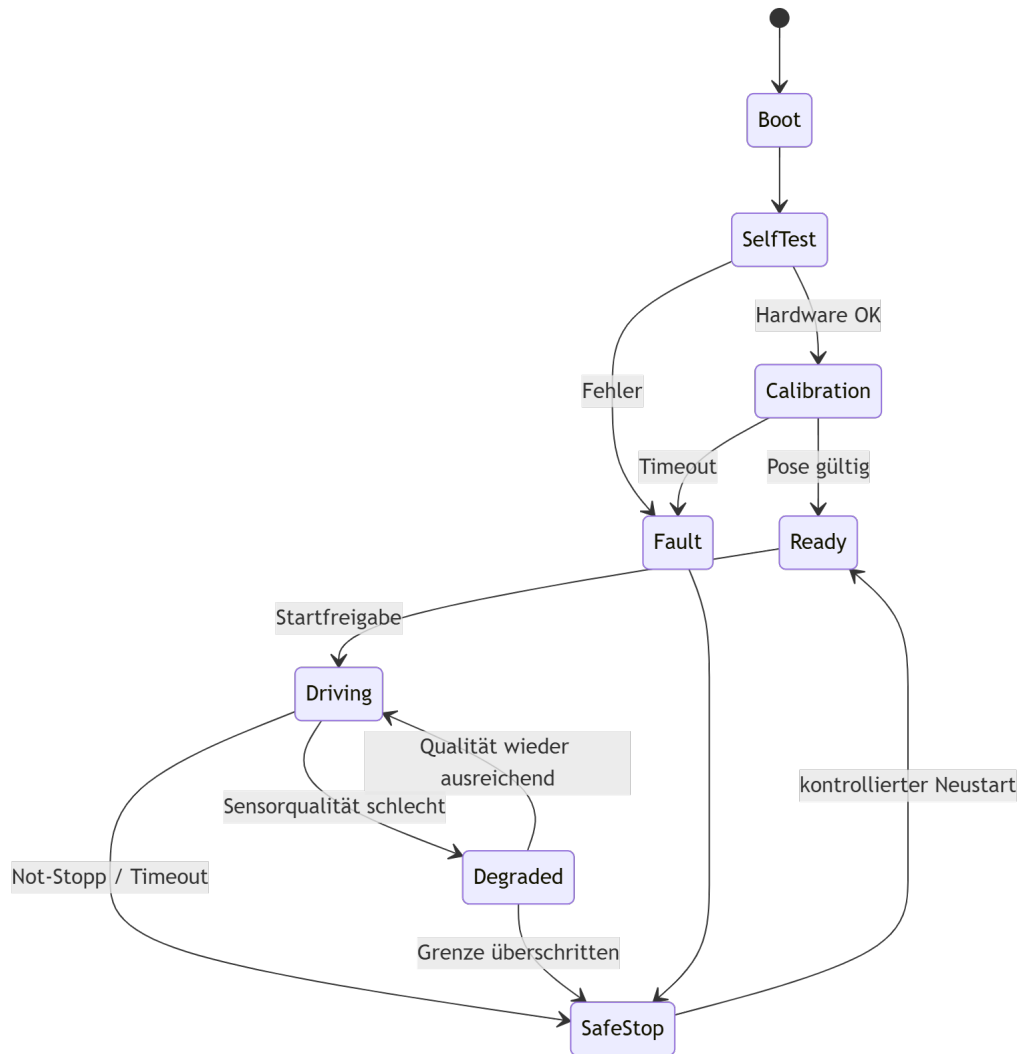
Die Policy darf Sollwerte vorschlagen, aber eine klassische deterministische Schicht prüft:

- maximalen Lenkwinkel,
- maximale Lenkänderung pro Zyklus,
- maximale Geschwindigkeit,
- Beschleunigungsgrenzen,
- Gültigkeit und Aktualität der Sensordaten,
- Freigabezustand und Not-Stopp,
- Timeout der Kommunikation.

Damit ist die ML-Komponente **nicht die alleinige Sicherheitsinstanz**.

Kapitel 8 – Zustandsautomat für den realen Betrieb

Die aktuelle Kalibrierlogik und die eigentliche Fahrt sollten auf realer Hardware als expliziter Zustandsautomat modelliert werden.

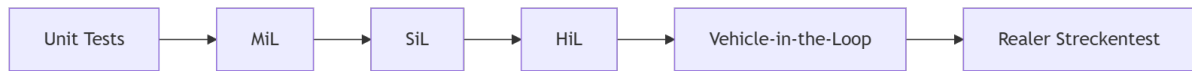


Warum dieser Punkt wertvoll ist

- RL wird nur im Zustand **Driving** verwendet.
- Kalibrierung und Selbsttest sind explizit getrennt.
- Fehlerbehandlung ist kein ungeordneter Block aus `if`-Abfragen.
- Statecharts könnten bei Bedarf parallele Regionen für Kommunikation, Diagnose und Fahrmodus modellieren.

Kapitel 9 – Vom virtuellen Test zur realen Absicherung

Testleiter



9.1 Unit Tests

Geeignete Funktionen aus deinem Projekt:

- Koordinatentransformationen,
- Ackermann-Winkel,
- Normalisierung und Clipping,
- Sortierung der Hindernisse,
- Winkel-Wrapping,
- Modellinferenz mit festen Referenzvektoren.

9.2 MiL/SiL-nahe Simulation

- viele zufällige Hindernispositionen,
- unterschiedliche Reibwerte und Fahrzeugparameter,
- Sensorrauschen und Verzögerungen,
- gezielte Ausfälle,
- Regression gegen gespeicherte Szenarien.

9.3 HiL

Die echte Zielhardware führt dieselbe Firmware und Policy aus, während Webots die Strecke und gegebenenfalls Sensor-/Aktorwerte simuliert.

Zu prüfen sind:

- echte Laufzeiten und Jitter,
- Kommunikationslatenzen,
- Speicherverbrauch,
- Watchdog-Verhalten,
- Timing unter hoher CPU-Last,
- numerische Unterschiede zur PC-Referenz.

9.4 Schrittweise reale Tests

1. Räder angehoben,
2. langsame Geradeausfahrt,

3. Kurven ohne Hindernisse,
4. einzelne statische Hindernisse,
5. komplette Strecke mit begrenzter Geschwindigkeit,
6. volle Geschwindigkeit und Störfälle.

CI/CD-Bezug

Eine Embedded-CI-Pipeline könnte:

- Python-Unit-Tests ausführen,
- Formatierung und statische Analyse prüfen,
- Firmware cross-kompilieren,
- Modell-Hash und Versionskompatibilität prüfen,
- SiL-Szenarien automatisch ausführen,
- auf einem angeschlossenen HiL-Prüfstand testen,
- nur signierte und getestete Firmware beziehungsweise Modelle ausliefern.

Kapitel 10 – Dependability und Fehlertoleranz

Erzählziel

„Fährt meistens“ reicht für ein reales eingebettetes System nicht aus.

Fehler – Störung – Ausfall am Projekt erklären

- **Fehlerursache:** LiDAR-Stecker locker oder fehlerhafte Speicheroperation.
- **Störung im Systemzustand:** veraltete oder unplausible Hinderniskarte.
- **Ausfall des Dienstes:** falsches Ausweichmanöver oder Kontrollverlust.

Geeignete Schutzmaßnahmen

- **Watchdog:** Neustart oder Safe Stop, wenn der Kontrollzyklus hängen bleibt.
- **Heartbeats:** Linux-Rechner und MCU überwachen sich gegenseitig.
- **Timeouts:** alte Sensordaten und alte Sollwerte werden verworfen.
- **Plausibilitätsprüfung:** Geschwindigkeit, Lenkwinkel und Pose müssen physikalisch möglich sein.
- **Redundanz:** Encoder plus IMU; eventuell verschiedene Verfahren zur Geschwindigkeitsbestimmung.
- **Informationsredundanz:** CRC, Sequenznummer und Modell-Hash.
- **Zeitredundanz:** Messung wiederholen, sofern die Deadline es zulässt.

- **Degraded Mode:** bei Verlust der Kamera langsamer fahren oder kontrolliert stoppen.
- **Assertions:** interne Invarianten während Entwicklung und Test prüfen.
- **Logging:** Ursache von Deadline-Verletzungen und Sensorfehlern nachvollziehen.

Dependability-Begriffe

- **Reliability:** Wie häufig fährt das System einen Lauf ohne Fehler?
- **Availability:** Wie schnell ist es nach einem Fehler wieder einsatzbereit?
- **Safety:** Verhindert es gefährliche Aktorbefehle?
- **Integrity:** Sind Firmware, Konfiguration und Policy unverändert und gültig?
- **Maintainability:** Können Sensor, Board oder Modell ausgetauscht werden, ohne das gesamte System neu zu schreiben?

Empfohlene Schlussaussage

„Der Sim-to-Real-Transfer besteht nicht nur darin, das ONNX-Modell auf einen kleineren Rechner zu kopieren. Die eigentliche Embedded-Softwareentwicklung beginnt mit der Definition von Schnittstellen, Zeit- und Speicherbudgets, der Trennung von komplexer Autonomie und deterministischer Sicherheit sowie einer stufenweisen Verifikation von SiL über HiL bis zum Fahrzeug. Meine RL-Policy bleibt dabei der Planer – erst Hardwareabstraktion, Echtzeitarchitektur, Tests und Fehlertoleranz machen daraus ein verlässliches eingebettetes System.“

Verbindung zu den vier Vorlesungsblöcken

Vorlesungsblock	Einbindung in den roten Faden
I – Hardware-Progression	Zielplattform, Mikrocontroller versus Mikroprozessor, Speicher, FPU, GPIO, ADC, Timer, Interrupts, DMA, UART/SPI/I ² C/CAN, HAL/BSP
II – RTOS und Scheduling	Super Loop versus Tasks, 100-ms-Deadline, WCET, Jitter, RMS, Queue, Semaphor, Mutex, PIP/PCP, FreeRTOS
III – Software-Methodik	modulare Architektur, Zustandsautomat, MiL/SiL/HiL, Unit Tests, CI/CD, statische

	Analyse, Watchdog, Dependability und Redundanz
IV – Linux, Multicore und Edge AI	Embedded Linux, PREEMPT_RT, AMP-artige Aufteilung, ESP32-S3/SMP, TinyML, TFLite Micro, Quantisierung und optional OTA

Priorisierung: Was unbedingt hinein sollte

A-Priorität – Kern der Geschichte

1. aktuelle Vier-Stufen-Pipeline,
2. konkrete Sim-to-Real-Lücken,
3. 100-ms-Deadline und WCET,
4. Super Loop versus RTOS-Tasks,
5. HAL/BSP und austauschbare Simulations-/Hardwaretreiber,
6. Embedded Linux versus MCU versus heterogene Architektur,
7. ONNX, Quantisierung und Ressourcenbudget,
8. MiL/SiL/HiL,
9. Watchdog, Safe State und Sicherheitskäfig um RL.

B-Priorität – zur Vertiefung

- Interrupt, ISR und DMA,
- Queues, Semaphore und Prioritätsinversion,
- Statechart,
- CAN/UART-Kommunikation,
- CI/CD und statische Analyse,
- Domain Randomization und Fault Injection,
- PREEMPT_RT oder SMP.

C-Priorität – eher für Rückfragen

- Harvard versus Von-Neumann,
- RISC versus CISC,
- Pipelining und Register,
- CCL, Event System und PORTMUX,
- Register Windows,
- OTA und Buildroot/Yocto im Detail.

Diese Begriffe sind nicht unwichtig, würden aber bei erzwungener Einbindung den roten Faden schwächen. In einer mündlichen Prüfung ist **begründete Tiefe** meist überzeugender als ein Begriffsfeuerwerk.

Eine besonders gute Prüfungsstrategie

Nutze in jedem Kapitel dieselbe Mini-Struktur:

1. **Problem im Projekt:** „In der Simulation erhalte ich Geschwindigkeit direkt aus Webots.“
2. **Embedded-Konzept:** „Real brauche ich Timer/Encoder, Interrupts und eine Zustandsabschätzung.“
3. **Entwurfsentscheidung:** „Die Messung läuft periodisch auf dem MCU und wird per Queue bereitgestellt.“
4. **Nachweis:** „Im HiL messe ich Latenz, Jitter und Verhalten bei Sensorausfall.“

So zeigst du nicht nur, dass du Begriffe kennst, sondern dass du sie **anwenden und gegeneinander abwägen** kannst.

Wahrscheinliche Rückfragen, auf die der rote Faden vorbereitet

- Warum ist ein Mikrocontroller und nicht nur ein Raspberry Pi sinnvoll?
- Ist das System harte oder weiche Echtzeit?
- Wie würdest du die WCET bestimmen?
- Warum reicht die durchschnittliche Inferenzzeit nicht?
- Welche Aufgaben gehören in eine ISR?
- Wo würdest du DMA einsetzen?
- Warum eine Queue statt einer globalen Variablen?
- Wie kann Prioritätsinversion entstehen?
- Was bleibt bei der Portierung unverändert?
- Was gehört in HAL und BSP?
- Wie testest du, dass das quantisierte Modell gleich reagiert?
- Was passiert, wenn die Policy NaN, einen Extremwert oder zu spät einen Wert liefert?
- Was passiert bei Ausfall von LiDAR, IMU oder Linux-Rechner?
- Wie unterscheidet sich SiL von HiL in deinem konkreten Aufbau?
- Welche Komponente darf den Motor im Fehlerfall tatsächlich abschalten?

- Warum ist RL nur ein Teil und nicht die gesamte Fahrzeugsoftware?

Quellenbasis

- [Vorlesungsglossar „Softwareentwicklung für eingebettete Systeme“](#)
- [Projekt-Repository](#)
- [RL Training Guide](#)
- [Beschreibung der Vier-Stufen-Architektur](#)
- [Aktueller Webots-Controller](#)
- [Policy- und Beobachtungslogik](#)